FanCode Gap Receipt and Dream11 Watch Along - Technical Document

- FanCode Gap Receipt & Dream11 Watch Along
  - A bounded multi-agent system for what a fan actually loses during a live match
  - 1. The fan problem
  - 2. The product
  - 3. Architecture
  - 4. The central design rule: the model proposes, code commits
  - 5. Multi-agent mechanics
  - 6. The evidence chain in Pack B, step by step
  - 7. AI tools and models
  - 8. Deterministic replay: the cassettes
  - 9. Product surface: HTTP, SSE, and bytes the server counts
  - 10. Evaluation and proof
  - 11. Limits, cost, and what changes in production
  - 12. Running it
  - 13. What this design is for

FanCode Gap Receipt & Dream11 Watch Along

A bounded multi-agent system for what a fan actually loses during a live match

Technical document — `dre11new`, November 2026. Everything below describes code that is in this repository. File paths, function names and measured numbers are real; where a claim is an intention rather than an implementation it says so.

---

1. The fan problem

A cricket fan on a phone during a live match fails in two very different ways, and this submission ships an artifact for each.

Failure one: the interval that disappears. An ad lands on top of a chase, the stream stalls for eleven seconds, or the fan joins at 15.1 because the commute did. They come back to a score that has moved and a situation they cannot read. Today they have exactly three bad options: scroll a commentary timeline that is 200 lines of text, watch a highlight reel that **spoils the very ball they missed**, or ask a chatbot that confidently invents “India needed 27 off 14” from a match it never saw. The information they need is tiny — *which balls, what happened, what is the situation now* — and the cost of getting it wrong is not neutral: a wrong required rate changes what a fan does on Dream11 that minute, and a spoiler ruins the ball they were about to watch.

Failure two: the creator claim nobody checks. Dream11’s engagement motion is creator-led video. A creator says “Kohli has 3 fours in this over, this is a must-watch chase” and 40,000 fans repeat it as fact. The platform knows the true ball-by-ball numbers and has no way to put them next to the claim at the moment the claim is made. And every engagement mechanic that sends a fan to video to find one number charges them data they never agreed to spend.

Both are the same engineering problem wearing two hats: **a language model is being asked to describe a live match, and a language model is not a scoreboard.** The answer is not a better prompt. It is a system where the model’s agency is real but bounded, and every fact on the screen is licensed by evidence the code itself can point at.

---

2. The product

One kernel, two packs, one deployment. A fan opens a link, picks a scenario, and watches an agent work; a toggle opens the **jury pane** that shows every model turn, tool call, budget line and refusal the run produced.

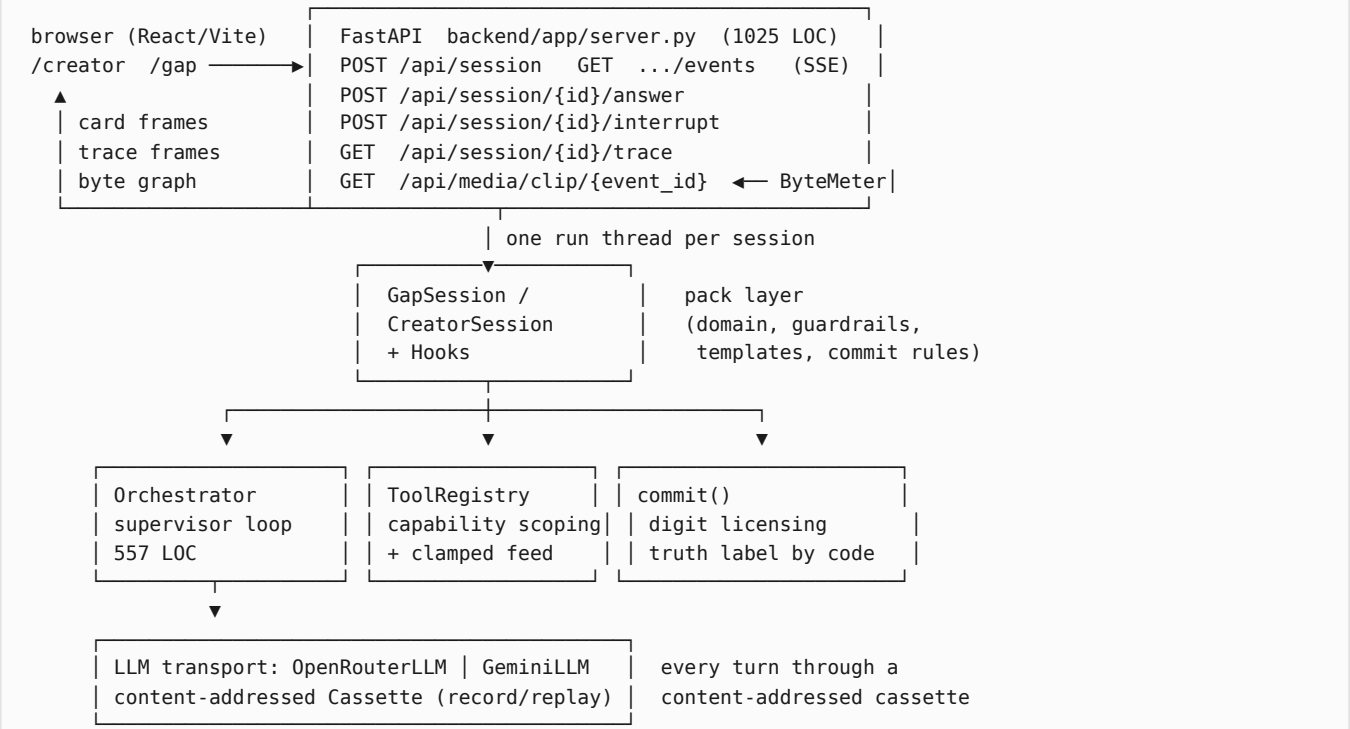
Pack B — FanCode Gap Receipt (`backend/packs/gap/`). The fan returns from an interruption. The system estimates *where they lost the stream* from their own player heartbeat log, decides (deterministically) whether one question to the fan would actually change the answer, reconstructs only the missed interval with evidence, and commits a **receipt**: the balls, the score change, the required-rate shift, what the scoreboard pixels said, an optional 12-second clip that loads only on tap — and a hard guarantee that nothing at or after their resume position is ever shown. If nothing key was missed, the receipt is an explicit no-op. Silence is not an available outcome.

Pack A — Dream11 Watch Along (`backend/packs/creator/`). A creator's Hinglish audio is transcribed, resolved ("*yeh over*"), classified, and each claim is checked against the feed the fan has already seen. Verified claims become verified cards; contradictions become conflict cards showing both the claim and the feed; predictions are logged in a ledger that settles only against balls that have actually been bowled.

The two packs share the kernel, the schemas, the commit gate, the cassette store, the trace format and the HTTP surface — they differ in epistemics. Pack A verifies a claim that already exists; Pack B has to *establish what the fan did not see* from a player log that may be lying.

---

3. Architecture



layer	files	responsibility
kernel	`backend/kernel/orchestrator.py` (557), `registry.py` (155), `commit.py` (158), `llm.py` (332), `openrouter.py` (244), `schemas.py` (386), `trace.py` (52), `clamp.py` (88)	agency mechanics, capability scoping, evidence licensing, replay, budgeting
pack B	`backend/packs/gap/session.py` (528), `boundary.py`, `voi.py`, `receipt.py` (201), `guardrails.py`, `ocr.py` (148), `rates.py`, `tools.py` (244), `agents.py` (124), `domain.py`	where the gap is, whether to ask, what the receipt may contain
pack A	`backend/packs/creator/session.py` (619), `verify.py` (289), `predictions.py` (227), `templates.py` (107), `tools.py` (249), `agents.py` (117)	what the creator claimed and what the feed says about it
transport	`backend/app/server.py` (1025), `backend/metering/bytes.py` (131), `backend/adapters/asr.py`	SSE journal, blocking asks, byte metering, run modes
evidence	`backend/fixtures/`, `backend/packs/*/domain.py`, `tests/cassettes/`	fixture feed, player telemetry, scoreboard frames, recorded model turns

The backend is 10,105 lines of Python across 60 files (`backend/**/*.py`); with `tests/` and `evals/` the repo is 13,868 lines. The full suite runs with no network: **252 passed, 1 xfailed in 3.87 s** (`python -m pytest tests -q --durations=3`; the single slowest test is 0.45 s, so

nothing here waits on a provider — and the xfail is the deliberate one described in §6 and §10, not a regression). The React front end is built and committed: `frontend/dist/` holds `index.html` (483 B), one hashed JS/CSS pair and the creator WAV set, and FastAPI serves it at `/`.

---

4. The central design rule: the model proposes, code commits

> **The LLM decides *what to look at* and *what it means*. Deterministic code decides *what is true, what it is called, how it is worded* and *whether it may be shown at all*.**

This is not a safety slogan; it is a set of concrete seams, each in a named file.

Numbers are licensed, not written. `kernel/commit.py` refuses a card whose printed digits do not appear in the evidence it cites — the cited events, a deterministic tally, a verdict, or an evidence note recorded by a tool at the moment it ran. A receipt that says "Required rate 8.48 → 11.57" is only possible because `rates.change()` computed those numbers and the tally text that licenses them (`gap_S3_evt_060_evt_075`) is on the citation list. A model that hallucinates a rate gets a rejection, not a warning.

Truth labels are never copied from the model. A `CardDraft` has no authority to declare itself `VERIFIED`. `commit()` assigns `TruthLabel` from the deterministic verdict through the pack's `LabelRule`, so the badge on screen is a property of the evidence, not of the model's confidence.

The wording of the cards that must be right is templated. `packs/creator/templates.py` is explicit: "The LLM still decides *whether* these happen and *what* the creator meant; it does not get to write the wording of a verified, conflict or settlement card."

A model cannot reach past the fan's cursor. `registry.py` hands each agent only the subset of the feed its capabilities allow through `ClampedFeed`. "No future events" is enforced by the clamp, not by a sentence in a prompt. `guardrails.py validate_receipt` then checks it again on the way out, per cited event id, plus the window's end label, and refuses with `spoiler_beyond_resume_cursor`.

A deterministic decision is *performed* by code, not offered to the model. This is the most interesting corollary and the one that is easiest to get wrong. If the value-of-information test says "asking is worth it", the kernel must be the thing that asks. `GapSession._kernel_ask` runs before the first supervisor turn. The docstring states the reason: if asking were only reachable through a model tool call, a run where the provider is unavailable would silently skip the fan — and the receipt would then be built on a boundary the fan never confirmed, which is the exact failure the question exists to prevent. The same rule covers "a card is owed": `_finish()` marks an episode *owed* when it ends `EXHAUSTED`, `FAILED`, or `SKIPPED` with the kernel's own must-commit flag, and only then builds the deterministic fallback.

VOI: asking is a measured property, not a vibe. `packs/gap/voi.py` computes, for every candidate boundary, the set of *material* events the fan would have missed, and asks only when two candidates produce different receipts — it looks for the `distinguishing_event`. Two boundaries that miss the same wickets are the same receipt, so the agent stays quiet and `no_question` is recorded with the reason. `CONFIDENT_ABOVE = 0.80` lets a confident telemetry read stand alone. Because this is code, the A/B is reproducible: `tests/test_gap_pack_b.py` runs the same scenario with two different fan answers and asserts the bodies differ while the headline and the cited tally stay identical — i.e. the question bought a real difference.

Grounding rule in Pack A. `creator/verify.py`: every number the analyst extracts must *literally appear in the utterance text*. If it does not, the claim becomes `UNVERIFIABLE` rather than being checked against a number the model invented. The model cannot create a fact by mishearing it.

question	decided by code	decided by the model
Where did the fan lose the stream?	`boundary.estimate` from heartbeat kinds	reads the candidates, never invents one
Should we ask the fan?	`voi.should_ask`	— (and the kernel asks, not the agent)
Which balls may be shown?	`ClampedFeed`, `validate_receipt`	chooses which to *narrate*
What do these balls mean for the chase?	`rates.change` computes the numbers	writes the one causal sentence
Is the claim true?	`verify.py` verdict vs feed	extraction, reference resolution, classification
What is this card called?	`commit()` + `LabelRule`	—
Is it worth the fan's attention now?	—	`strategist` modality choice, vetoable
Which tool next, which specialist?	bounded by budget + capabilities	supervisor `dispatch` / `tool` / `commit` / `skip`

---

5. Multi-agent mechanics

`orchestrator.py` runs a supervisor whose entire output surface is one JSON object (`SupervisorDecision`):

```
"action": "dispatch | tool | ask_user | request_permission | commit | skip | stop"
```

with an optional `agent`, `tool`, `args_json`, `question`, `options` and `card`. Each dispatch starts a **specialist** whose scope is enforced, not requested: a `SpecialistSpec` carries a tool name list and a capability set, and the registry refuses a call outside it — the refusal is *traced*, never swallowed. Pack B's roster is `reader` (playback log → boundary hypotheses), `investigator` (cross-check the window against frames and commentary), `questioner`, `strategist` (modality + the one causal sentence) and `critic` ("reject a receipt that would spoil or mislead the fan"). Pack A's is `perceiver`, `analyst`, `verifier`, `strategist`, `critic`.

Pack B supervisors are capped to the capabilities `{telemetry, past_events, video_evidence, ask_user, permission}` — the supervisor can look at everything but cannot mint a card; only the commit gate can do that.

`Hooks` is how a pack tells the kernel what its domain means, and every hook exists because of a specific failure:

hook	what it does	why it is a hook
`mandatory_check`	an unsippable validation on a tool result	a model can route around optional checks; it cannot route around this one
`before_commit`	rebuilds the receipt from the clamped feed before the draft is judged	the draft the model wrote is not the artifact the fan gets; the *feed* is the authority
`build_fallback`	deterministic draft when the run produced nothing	the fan is owed an outcome even at 0 valid model turns
`veto_terminal`	refuses a pack's own `skip` once	"nothing to do" is a decision a jury must see challenged
`wait_for_answer`	blocks on a real human answer	human-in-the-loop over SSE, with a per-session timeout
`observation_context`	injects domain notes into the prompt	keeps pack knowledge out of kernel code

`require_critic_before_commit=True` is the wall: a `commit` action that arrives before the critic specialist has run is refused. Budget accounting (`Budget`: `max_llm_calls`, `max_tool_calls`, `max_ms`, `max_specialist_rounds`) is enforced *before* a call is made, and the refusal is recorded as `budget_block`. The recorded live-demo HTTP run of S1 (`tmp/http_S1.jsonl`, 30 journal frames, `run_mode: live-demo`) consumed all 14 of its `max_llm_calls`, took one `budget_block` at `where: agent:critic` at seq 22, ran 102.9 s of wall clock, and still committed one card — the budget stopped the model, not the product.

The pack's promise, implemented. Pack B's rule is that a fan gets a receipt or an explicit no-op — never silence. Two bugs in that promise were found by driving the *real HTTP path* (a browser hits things an in-process runner cannot) and both are now closed, each with a named regression test in `tests/test_gap_pack_b.py`:

1. *The double ask.* The kernel asked, the fan answered, and then the supervisor asked its own second question. Nobody was left on the line to answer it, the episode ended `awaiting_user`, and the fan received **nothing**. `GapSession._wait_for_answer` now short-circuits when the boundary is already settled — it returns the answer on file and records `ask_refused`, instead of blocking a second time (`backend/packs/gap/session.py:425-430`). The ask tool already had this guard; the *hook path* did not, which is why it survived unit testing and only appeared over SSE. Covered by `test_the_fan_is_never_asked_twice` (line 296) and `test_a_model_question_after_the_kernels_does_not_bother_the_fan` (line 396), the latter driven by a supervisor stub whose docstring names the bug it reproduces: "A supervisor that only ever asks a question - the shape of the HTTP bug" (line 370).
2. *The silent end.* `_episode()` now commits the deterministic receipt whenever a scenario ends with no card, whatever the terminal reason — exhausted, failed, vetoed, or waiting on an answer that never came. The receipt it builds cites only the clamped feed, so it cannot spoil and cannot invent; the trace says which wall it fell back from (`note: "deterministic receipt after {reason}"`, `session.py:240`). Covered by `test_an_unanswered_question_still_sends_the_deterministic_receipt` (line 409) and, off the model entirely, `test_no_model_still_asks_the_fan_and_commits_a_receipt` (line 224) and `test_the_off_layer_commits_a_receipt_without_a_single_model_turn` (line 244). All three pass in the offline run: `python -m pytest tests/test_gap_pack_b.py -k "never_asked_twice or does_not_bother_the_fan or unanswered_question" -q` → **3 passed, 25 deselected** (1.03-1.24 s across the two runs on this machine).

The ask timeout moved to the pack's own config (`GapConfig.ask_timeout_s`) because the kernel hard-coding 40 seconds meant a web session could hold an episode open for a fan who had already answered; `WebGapSession` raises the config value for real human patience (90s in the container).

---

6. The evidence chain in Pack B, step by step

A run of S1, in the order things actually happen:

1. **Fan returns.** `run_scenario("S1")` resets per-scenario state — the boundary, the released event set, frame checks, clip quotes, tallies, evidence notes and the fan's answer — so scenario N cannot inherit scenario N-1's receipts.
2. **Boundary from the player, not the feed.** `boundary.estimate` reads the heartbeat log. The kinds matter: `play` is *VIEWED*, `ad_start | stall | gap` is *INTERRUPT*, `ad_end | resume | join` is *restart*. A late joiner with no rendered ball at all yields no boundary — the telemetry genuinely cannot say, and that is exactly when asking is worth it (S3). Output is a `Boundary`: candidate last-seen balls plus a confidence and notes. Measured for S1: `candidates=['16.2', '16.1', '16.3'] conf=0.65`, with the notes "last ball the player rendered: 16.2 (play)" and "ad overlay covered the screen over ball 16.2: boundary widened by one ball either side, confidence -0.3".
3. **VOI gate + kernel ask.** `voi.should_ask` compares the receipts each candidate would produce. For S1 the answer changes which balls are inside the window, so the kernel poses the canonical question — *"Did you see the wicket at ball 16.3 (142/4) before you left?"* with options `['Yes, I saw ball 16.3', 'No, I left at 16.2']` — and blocks on the HTTP `answer` endpoint. `_apply_answer` parses an over label out of the fan's words with a regex and accepts it **only if it is one of the candidates**; a non-candidate answer never moves the boundary (tested), and an answer pointing past the resume cursor is recorded as uncertainty and ignored. The accepted answer becomes evidence (`fan:<event_id>`), the badge source flips to `boundary: fan`, and the `boundary_from_fan` event lands in the trace.
4. **Supervisor turn 1.** The prompt it sees contains the observation, the domain notes, the settled fan answer and the remaining budget — with **no wall-clock in it** (see §8). It dispatches `investigator`.
5. **Tools.** `query_event_feed` (clamped), `search_commentary`, `inspect_frame`, `offer_clip`, `ask_fan`, `get_playback_telemetry`. Each returns a structured dict and records evidence notes at the moment it ran. `inspect_frame` on S1 returned, measured from the trace, `{"found": true, "frame_id": "frm_16_3", "path": "frames/score_16_3.png", "read_from_pixels": {"score": "142/4", "over": "16.3", "rrr": "9.71", "readable": true}, "feed_score": "142/4", "agrees": true}` in 54.8 ms, which `session.note_frame_check` turns into the evidence note `frame:16.3 → "frm_16_3 at 16.3 read from pixels: score 142/4 over 16.3 rrr 9.71; feed says 142/4"`.
6. **Pixels, not a lookup table.** `gap/ocr.py` reads the scoreboard out of the rendered frame's pixels. The docstring is worth quoting because it names the temptation this module resists: "`frames.json` already knows each frame's true score, so the honest version of 'inspect the frame' could have returned that string and the demo would still pass — while proving nothing. The injected-fault frame is the test that separates the two: it is rendered with a score that is *not* in the feed, and a real reader must report 143/4 there so the verifier can reject the clip." When pixels disagree with the feed, `offer_clip` returns `offerable: False` and the trace records `clip_rejected` — the clip is refused *because the evidence chain broke*, which is the whole point of having one. Measured today: `test_clip_is_refused_when_the_pixels_disagree` passes, and the refusal string it asserts against the fault ball is "frame shows **143/4** but the feed says **146/4**" (`tests/test_gap_pack_b.py:175`), while `test_a_frame_that_agrees_is_a_real_check` proves the reader pulls `142/4` out of `frm_16_3`'s actual pixels. A clip is only ever quoted, never pushed: `quote["bytes_estimate_kb"] == 12 * 200 = 2400` KB with `load_policy: "not fetched until the fan taps it"`. One honest wrinkle, and the suite names it: the run's *only* xfail (`tests/test_acceptance_gap.py:338-343`, `strict=True`, test body at line 344) documents that `clip_is_offerable` reads a top-level `readable` that only `GapSession.frame_check_for` sets, while `inspect_frame` nests it under `read_from_pixels` (`backend/packs/gap/tools.py:142`). Consequence in the live S1 journal: even though the reader reported `142/4 == feed`, the clip came back `{"offerable": false, "why": "frame unreadable by the scoreboard reader"}` and the receipt says "No clip loaded". The refusal is therefore conservative — it never lets an unverified clip through — but it is a real defect, not a designed outcome.
7. **Strategist.** Chooses the modality (`receipt` / no-op / clip) and writes the one causal sentence. `before_commit` rebuilds the receipt from the clamped feed regardless of what it asked for; the strategist's sentence is then re-checked by `validate_receipt`, which refuses a causal claim no evidence supports.
8. **Critic, then commit.** `require_critic_before_commit` is what makes a commit-without-review impossible. `commit()` licenses the digits, assigns the label, enforces the caps (headline 90, body 700 characters), stamps the source badges, and appends the card. A rejection is recorded with its violation list so the failure is visible in the trace.
9. **The receipt.** S1 as committed offline by `python backend/scripts/run_pack_b.py --llm replay --budget 14` — label `VERIFIED vs feed`, 36-character headline, 599-character body, 14 llm calls:

```
You missed 16.3-16.6: 142/3 -> 147/5
- 16.3 WICKET: WICKET! Vikram Naidu bowled through the gate (142/3 -> 142/4)
- 16.4 BOUNDARY: FOUR! Kabir Menon answers straight away (142/4 -> 146/4)
- 16.6 WICKET: RUN OUT! Kabir Menon short of the crease. Two... (146/4 -> 147/5)
Score 142/3 -> 147/5. Required rate 9.27 -> 9.67 (+0.4); 29 needed off 18 balls.
Other balls in the window: 16.5
Why it matters: You missed a wicket at 16.3 and a four at 16.4 while you were away.
Scoreboard frames read from the pixels: 16.3 read 142/4 == feed.
No clip loaded - this receipt is text.
Nothing after 16.6 is included.
cites e=['evt_068', 'evt_069', 'evt_071', 'evt_070'] t=['gap_S1_evt_067_evt_071']
```

Every digit above is licensed by `gap_S1_evt_067_evt_071` or by a cited event; the live HTTP run of the same scenario (`tmp/http_S1.jsonl`, 14 llm calls, 102.9 s) committed the identical headline and the identical score/rate lines, with the strategist's one causal sentence worded differently — which is the division of labour this section exists to describe.

The fit ladder in `receipt.py`. A 15-ball window cannot be 15 bullets on a phone, and a receipt that breaks the 700-character body cap is a *rejected commit* — meaning the fan gets nothing. So the template is designed around that contract: `CARD_LIMIT = 690` (headroom for the joins), `MAX_BULLETS = 5`, `key_priority()` selects by consequence (wickets, then boundaries, then over order) while `bullet_block()` prints in feed order so a collapsed window still reads forwards in time, `assemble()` drops *optional prose from the bottom up* before a key-action line is ever cut, and the non-key balls degrade from an explicit list → one digit-free sentence → absent, with the tally carrying the arithmetic. Digits only ever live in required lines, so the collapse can never corrupt a number. S3 (15 balls) commits at 690 or fewer characters.

---

7. AI tools and models

Text backend. Two interchangeable transports behind one contract (`complete(prompt, system, schema, ...) -> (parsed_model, meta)`): `kernel/llm.py` (Gemini) and `kernel/openrouter.py`. OpenRouter exists for a specific, boring reason stated in its own docstring: the Gemini key this project was built against is free-tier and caps `generate_content` at 20 requests/day, which is less than one 12-segment episode needs — so "the LLM was unavailable" would otherwise become the reason a demo cannot show the loop. Structured output is requested as `json_object` plus the schema written into the system prompt rather than strict `response_format`, because the pydantic-validated retry in the orchestrator already catches a bad shape and a stealth-tier model is free precisely because it is under-specified.

Model chain and resilience. `OPENROUTER_MODEL` (default `stealth/crouton-alpha`) plus `OPENROUTER_MODEL_CHAIN` — a comma-separated list of free-tier models tried in order, with the primary inserted at the head. Per call: 45 s timeout; `finish_reason == "length"` retries the *same* model with 1.8× the output budget (a long `thought` field truncating the JSON is the most common failure, and it is a budget problem, not a model problem); an empty completion — what a stealth model returns when it spends the whole budget on reasoning — gets one retry; HTTP 429 honours `Retry-After` when the delay is sane, 5xx retries with backoff. Every failed attempt is kept as a note and the whole chain's story appears in the single error if all of them fail, so the trace says *which model failed how* rather than "LLM error".

How the chain was chosen. `backend/scripts/bench_models.py` sends one realistic `SUP_CONTRACT` supervisor turn to each candidate and prints status code, latency, output tokens and whether the reply validated against `SupervisorDecision`. Measured: `stealth/crouton-alpha` VALID 6.1 s; `nvidia/nemotron-3.5-super-vl:free` VALID 2.0 s; `xingchen-agi/xing4.0-29b-a4b:free` INVALID (rejected its own `thought` length); `nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free` INVALID (dispatched an agent role that does not exist in the roster); several models returned no content at all at a 900-token budget, two hit 429, one timed out at 75 s. The chain is ordered by that table, and the invalid ones are *kept in the chain anyway* because an unparseable reply costs a retry and a dead provider costs the episode. Those latencies are one recorded bench pass, not a reproducible measurement — a free tier is variable by definition — so treat the VALID/INVALID column as the durable result and re-run the script before quoting a number from it. What *is* reproducible here: the model that served every turn of the live probe in §7.1 was `stealth/crouton-alpha`, read back out of the cassette payload it wrote.

Audio. `adapters/asr.py`: `GeminiASR` (real audio → text with keyterm boosting) and `CachedASR` (the fixture transcript, badged "ASR: cached"). The honest caveat is in the docstring: Gemini emits no acoustic confidence, so `confidence` is the model's self-reported word-level certainty, labelled as such in the UI and used only to gate abstention — never to assert a fact.

Badges, not claims. Every card carries the source badges of the run that produced it: `llm: live | cassette | unavailable`, `feed: replay_fixture`, `telemetry: simulated`, `frames: rendered_fixture`. `mode_used` starts at `"none"` and becomes `"unavailable"` on a cassette miss with live disabled. This detail matters: a badge that reports the *configured* backend rather than the one that *served the call* is a lie you tell your own demo, and the UI would then claim model turns that never happened. The badge reports what actually served each call, read back from the cassette payload.

7.1 Proof that the live path is a model and not a template

The obvious objection to a cassetted demo is "your replay is a lookup table wearing an LLM". Segment u02 (`Yeh banda ab reverse sweep try kar sakta hai, field dekho, third man up hai.`) answers it with two runs of the same input:

served by	command	badge	headline the card got
the cassette store	`python -u backend/scripts/run_pack_a.py --llm replay --only u02`	`llm: cassette`	Reverse sweep option **with** third man up
the provider	`LLM_CASSETTE_DIR=<empty dir> python -u backend/scripts/run_pack_a.py --llm live --only u02`	`llm: live`	Reverse sweep option **flagged** with third man up

The live run spent 8 provider calls, all served by `stealth/crouton-alpha`, and 7 of its 8 prompt keys were new — never recorded before. What did *not* change between the two runs is the part that matters: `TruthLabel.OPINION`, `kind=insight`, `tmpl=free_text`, `cites u=['u02']` and `numbers=[]` — the kernel licensed no digit because no digit was in evidence — 1 card, 8 llm calls, 10 trace events. One turn is an even tighter test than the headline: prompt key `8d6be6ed6ebd2aee44a4463b25a89a5a` is content-addressed over system+prompt+schema+temperature, so the live and stored requests are byte-identical. The stored reply's `thought` was "…based on field placement, needs classification before any card"; the live reply's was "…based on field with third man up, needs classification before any verdict or card", with a different `reason` — and both returned `action: dispatch, agent: analyst`. The model's prose moved; the decision, the label and the licensing verdict did not.

There is a trap in here worth naming, because we hit it: `complete()` looks the cassette key up *before* it considers the provider (`kernel/openrouter.py:96-110`), even in `record` mode. The first `--llm live` run of u02 therefore reported `llm: cassette` and wrote nothing — a "live" demo that was really a replay. Pointing `LLM_CASSETTE_DIR` at an empty directory is what makes a live claim honest, and it is also what keeps a live pass from writing over the committed store: `Cassette.save` overwrites unconditionally when `mode == "record"` (`kernel/llm.py:121-128`).

---

8. Deterministic replay: the cassettes

A multi-agent demo whose AI cannot be re-run is a video, not a product. Every model turn is therefore recorded in a **content-addressed cassette store** (`tests/cassettes/`, committed on purpose; **585 entries** at the time of writing, and `/api/health` reports the count live so a reviewer can check the number instead of trusting this sentence). The count is not a constant: this store held 440 entries until the `record` half of the `compare_runs.py openrouter record replay` check below appended the 14 turns that were not yet on disk — which is exactly what a recording pass is for. A live pass aimed at the committed store can also overwrite it (§7.1).

`Cassette.key(system, prompt, schema_json, model, temperature)` → a hash; `complete()` looks the key up first. Three rules were learned the hard way, and each one is a real bug that is now fixed in code:

1. **Only a reply that parses is worth recording.** `parse_output` runs *before* `save`. A cassette containing truncated JSON would replay the failure on every later run — one bad recording becomes a permanent outage. Symmetrically, an entry that fails to parse *on replay* is `drop()`ped and re-recorded, because a stale entry is worse than no entry.
2. **Prompts must be run-independent.** The supervisor prompt used to include the remaining budget, which contained `ms` — a wall-clock number. Every run therefore produced a unique cassette key, replay missed 100% of the time, and the offline path silently became `llm: unavailable` with 3 model turns instead of 14. Fix: `ms` is dropped from the `remaining` dict that is rendered into prompts. Proven by a record-vs-replay prompt diff: before the fix exactly one line differed; after it, prompts are byte-identical.
3. **A retry records under the base prompt.** When a reply fails validation the orchestrator re-asks with a correction nudge appended. That second prompt is the model's answer to the question it was actually asked, so `complete(..., key_text=<base prompt>)` records it under the original key. Without this, one 502 from a provider permanently removes that turn from the offline path.

`backend/scripts/compare_runs.py openrouter record replay` is the machine-proof of all of it: it compares the `commit` events of the two runs and fails if the card bodies, headlines or citations differ, if any card came from a deterministic fallback, or if the replay shows zero `supervisor_decision` / `tool_call` events. Result from this checkout (the `kinds` dict is re-wrapped for the page; the script prints it on one line, and the replay line is its own copy):

```
record: 3 cards | 59 events | kinds={'ask_user': 2, 'auto_answer': 2, 'boundary_from_fan': 1,
        'episode_context': 3, 'episode_start': 3, 'supervisor_decision': 14, 'tool_call': 15,
        'agent_result': 5, 'budget_block': 2, 'commit': 3, 'episode_end': 3, 'no_question': 1,
        'terminal_vetoed': 1, 'modality_overridden': 3, 'agent_no_final': 1}
replay: 3 cards | 59 events | kinds={ identical to the line above, key for key }
  receipt  You missed 16.3-16.6: 142/3 -> 147/5
  noop     Nothing key missed (147/5 unchanged by any wicket or boundary)
  receipt  You missed 15.2-17.4: 135/3 -> 149/5
MATCH: 3 identical cards, 14 model turns, 15 tool calls, 0 fallback commits
```

Note that `supervisor_decision: 14` appears on the *replay* side too: the supervisor's turns are cassetted exactly like the specialists', so the offline demo is not "real agents, invented supervisor" — the whole loop replays.

The same equivalence is proven one layer higher, over the real HTTP surface, with `backend/scripts/diff_frames.py`: two `drive_http.py` journals of the same scenario — one served live (`backend/out/http_S1_live2.jsonl`: `llm: live`, 28 frames, 14 llm calls, 98.5 s of journal span) and one replayed from the cassettes that call recorded (`backend/out/http_S1_replay2.jsonl`: `llm: cassette`, 28 frames, 14 llm calls, 0.6 s) — are compared frame by frame after dropping timestamps, ids and the provenance badge itself:

```
$ python backend/scripts/diff_frames.py backend/out/http_S1_live2.jsonl \
      backend/out/http_S1_replay2.jsonl
frames: 28 vs 28 | cards: 1 identical
MATCH: same trajectory, same cards, only provenance and wall-clock differ
```

For scale, the same measurement on the other recorded journals: `tmp/http_S1.jsonl` 30 frames / 102.9 s live, `tmp/http_S3.jsonl` 29 frames / 119.1 s live, `tmp/http_S2.jsonl` 27 frames / 0.2 s cassette — one ask, one card, 14 llm calls each.

Three run modes make the ladder explicit: `offline-ci` and `cached-demo` replay cassettes and never call out; `live-demo` calls the provider and records what it hears. `safe-demo` is accepted as a mode name but is not a distinct code path — `backend/app/server.py` computes `live = mode == "live-demo"` (line 107), so `safe-demo` behaves exactly like `cached-demo`. A "cassette first, live only to fill a miss" rung is a good idea that this build does not implement, and the README says so rather than letting the mode table imply otherwise. The replay runner reports **3 cards, 22 llm calls, 43 trace events** across S1/S2/S3 with zero network access; with the provider genuinely dead (`--llm off`) the same three scenarios still commit 3 cards in 19 events with **0 llm calls** and badge `llm: unavailable`, because the deterministic fallback answers for it.

---

9. Product surface: HTTP, SSE, and bytes the server counts

`backend/app/server.py` is one FastAPI app that serves the API under `/api`, the built React app at `/`, and the fixture media under `/api/media`. `docs/API_CONTRACT.md` is the only place the two halves agree.

- **One run thread per session, one journal per session.** Every kernel event is appended to a journal, and `GET /api/session/{id}/events` *replays the journal from the beginning* before following live. A browser that subscribes late — because the reviewer reloaded — still sees the whole run. This is not a convenience: an SSE-only design loses the frames emitted before the fan's answer POST landed, which is precisely the moment the run is most interesting.
- **Blocking human-in-the-loop over HTTP.** `POST /api/session/{id}/answer` sets a `threading.Event` the run thread waits on (`wait_for_answer` hook), bounded by the session's `ASK_TIMEOUT_S`. An unanswered question no longer ends the episode empty: it falls through to §5's never-silent rule.
- **`Watch` vs `Quiet` is server behaviour, not CSS.** `mode` is sent in `POST /api/session`. In quiet mode `/api/media/clip/{event_id}` refuses with `403 {"why": "quiet mode never loads clip bytes"}`. `metering/bytes.py` counts every byte the media route puts on the wire through `counted_stream`, per session, cumulative, never reset by a reload, and keeps both `total_bytes` (what was really served) and `would_have_kb` (what loading every offered clip would have cost). The numbers are code-fixed and test-enforced: a clip is quoted at `CLIP_SECONDS * BYTES_PER_SECOND_KB = 12 * 200 = 2400` KB, and `test_quiet_mode_never_pays_a_byte_and_watch_pays_exactly_the_clip` (`tests/test_acceptance_creator_http.py:406`, passing in this checkout's run) asserts the quiet refusal answers `charged_kb 0.0 / would_have_kb 2400` with `bytes_sent == 0` and `clip_loads == 0`, while the same fan in watch mode gets exactly `2400 * 1024 = 2,457,600` bytes and an `X-Clip-Kb: 2400.0` header — so "we only loaded what you tapped" is a server measurement, not a marketing claim. The clip actually priced in the live S1 run shipped 0 bytes because the clip was refused upstream (`done.bytes_kb 0.0`, `would_have_kb 0.0`, `clip_loads 0`). The docstring names the reason: *"the whole 'we only load the clip the fan actually asked for' claim is worth nothing if the client reports the byte count."*

- **Live interruption telemetry.** `POST /api/session/{id}/interrupt {kind}` appends a heartbeat row to the session's own player log, which the boundary estimator then reads — a jury can drag the boundary by stalling the player, and prove the boundary tracks the player rather than the feed.
- **Everything is observable.** `GET /api/session/{id}/trace` returns the events, badges, cards and budget used, and the same events stream to the journal as `supervisor_decision`, `tool_call`, `agent_result`, `guardrail`, `clip_rejected`, `terminal_vetoed`, `ask_user`, `boundary_from_fan`, `no_question`, `ask_refused`, `budget_block`, `commit`, `bytes`, `card`, `status`, `done`. The UI is a thin renderer over a journal that already contains the argument for the answer.

---

10. Evaluation and proof

command	what it proves	measured on this checkout
`python -m pytest tests -q`	every deterministic rule, the guardrails, and the two HTTP regressions	**252 passed, 1 xfailed in 3.87 s** (slowest single test 0.45 s), no network
`python -u evals/run_evals.py`	the four hard invariants across both packs	**PASS - all four hard invariants hold**: 4 runs, 11 cards, 36 model turns, 24 tool calls, 0 silence, 0 spoilers, 0 unlicensed digits, 0 fallbacks, `llm_badge=cassette` on all four rows
`python -u evals/run_evals.py --llm off`	the provider-day where nothing works	**PASS**: 4 runs, 5 cards, 0 model turns, 0 tool calls, 0 spoilers, 0 unlicensed, badge `llm: unavailable` on every row
`python -u backend/scripts/run_pack_b.py --llm off`	a fan still owes a receipt with the model dead	3 cards (S1/S2/S3), **0 llm calls**, 19 events, `llm: unavailable`
`python -u backend/scripts/run_pack_b.py --llm replay`	the recorded model turns replay	3 cards, 22 llm calls, 43 events, `llm: cassette`
`python -u backend/scripts/run_pack_a.py --llm replay`	Pack A's full agent path offline	8 cards, 58 llm calls, 23 supervisor decisions, 18 tool calls, `llm: cassette`
`python -u backend/scripts/compare_runs.py openrouter record replay`	replay *is* the recorded run	3 identical cards, 14 turns, 15 tool calls, 0 fallback commits
`python -u backend/scripts/drive_http.py --base http://… --pack gap --scenario S1 --answer "…" --log backend/out/http_S1_live2.jsonl`	the link a reviewer opens	live: 1 card, **1 ask**, 28 journal frames, 98.5 s, `llm: live`, 14 llm calls
`python backend/scripts/diff_frames.py backend/out/http_S1_live2.jsonl backend/out/http_S1_replay2.jsonl`	the hosted artifact is the same run	**28 vs 28 frames, 1 card identical** - `MATCH: same trajectory, same cards, only provenance and wall-clock differ`; replay side 0.6 s, `llm: cassette`
`python backend/scripts/make_space_bundle.py`	the deploy directory is self-contained	staged bundle measured on this checkout: **763 files, 29 MB, 585 cassettes copied, no `.env` shipped**
`python backend/scripts/bench_models.py`	which models are in the chain and why	validity + latency table in §7

`evals/run_evals.py` writes `evals/results.json`; those four rows, verbatim from the file this checkout produced, are the per-run detail behind the totals above:

pack / run	outcome	card kind	truth label	body chars	model turns	tool calls	llm calls	asked / answered	clip rejects	fallback	badge	s
gap S1	card	`receipt`	VERIFIED vs feed	599	6	7	14	1 / 1	1	false	cassette	0.12
gap S2	card	`noop`	Evidence-backed	192	5	6	14	0 / 0 (`no_question` 1, `terminal_vetoed` 1, `modality_overridden` 3)	0	false	cassette	0.06
gap S3	card	`receipt`	Evidence-backed	600	3	3	6	1 / 0	0	false	cassette	0.03
creator, 12 segments	8 cards	conflict, insight, prediction, settlement	Opinion 3, VERIFIED 1, Prediction-tracked 1, CONFLICTS 1, Prediction HIT 1, Creator-said 1 (9 commits, 1 rejected)	221 (longest of the 8; the gap rows are single cards, so their one body)	26	21	63	0 / 0	0	false	cassette	0.28

Two things that table is designed to make impossible: a card over 700 characters (the S3 window is 15 balls and lands at 600), and a gap scenario that ends without an artifact (`asked` is 1 for S1 and S3, and the unanswered S3 still commits).

The test suite is deliberately written so the *model-off* cases are load-bearing — the file that covers Pack B opens with "every assertion here is about a claim the fan can check… the model-off runs are the load-bearing ones, because a demo where the provider is down still owes the fan a truthful receipt." Coverage includes: the boundary is the last *rendered* ball and not the last ball the feed knows; the fan is never asked twice; a non-candidate answer never moves the boundary while a candidate answer does and lands as evidence; the two fan answers produce different bodies but identical headlines and identical cited tallies; no event past the cursor is ever released to any agent (`released_events` is asserted against the cursor index on every run); a receipt for a 15-ball window still fits 700 characters; a clip is refused when the pixels disagree; and the two HTTP bugs of §5 have named regression tests with a supervisor stub that only ever asks questions.

Three gaps I want to name rather than hide. First, the cassette store is a *recorded trajectory*: if a prompt is edited, the recording pass must be redone, or replay falls back (and says so in the badge). Second, the browser path and the in-process runner path historically generated different prompts, so a `cached-demo` browser session could miss turns that the runner had recorded; the recording pass is now driven *through HTTP* (`live-demo` + `drive_http.py`) so the cassettes cover the trajectory the UI actually takes. Third, the suite carries exactly one deliberate failure — `test_a_ball_the_investigator_already_checked_is_still_offerable`, marked `xfail(strict=True)` with the defect spelled out in its reason (the two writers of the frame cache disagree on where `readable` lives, `backend/packs/gap/tools.py:142`). It is strict, so the day someone fixes the shape mismatch the suite fails and forces the marker to be removed; it is left failing rather than deleted, because a reviewer deserves to know that a clip the investigator has already verified is currently refused. The refusal is in the safe direction.

---

11. Limits, cost, and what changes in production

- **Fixtures, not a live feed.** The ball-by-ball feed, the player heartbeat log and the scoreboard frames are synthetic and rendered by code this repo owns (`backend/fixtures/build_fixtures.py`, 456 LOC). That is the right call for a submission — the agent has to be able to be *wrong* about the evidence, which a pre-chosen answer would prevent — but in production the same interfaces take the real telemetry and stream pipelines, and `feed: replay_fixture` becomes `feed: live`. The badges exist precisely so this distinction is never hidden.
- **Free-tier providers are variable.** A 502, an empty completion or a truncated reply is routine, not exceptional. The mitigations are the chain, the length retry, the parse-before-save rule and the fallback ladder; the honest numbers, measured from the recorded live-demo HTTP journals, are a live S1 episode of **98.5 s** (`backend/out/http_S1_live2.jsonl`) at 14 llm calls — the same scenario replayed from the cassettes that call recorded takes **0.6 s** — and the two other live journals in `tmp/` span 102.9 s and 119.1 s. So "about 100 s and all 14 of the budgeted calls" is the number to quote, not the 0.2 s the offline demo shows. In production: pin a paid

model, cache the tool results per over (they are already pure functions of the feed), and run the critic in parallel with the strategist rather than in series — in the live S1 run the critic was the agent that met the wall (`budget_block`, `where: agent:critic`), and the receipt committed anyway.

- **Bounded cost per episode is a feature you can quote.** `Budget` caps llm calls, tool calls, specialist rounds and wall clock, and the cap is enforced before the call, so an episode cannot spiral. The card and the trace both show the consumption.
- **One match, one interruption per demo scenario.** S1 (ad over a wicket pair), S2 (a stall during a run chase), S3 (a late joiner with no rendered ball at all) exercise three different ways the evidence can be missing, not an exhaustive set.
- **Abstention is a designed outcome, and it is cheap to fake.** A no-op card is a valid result here — but the no-op is *computed* (`receipt.no_op` with the tally) and traced (`no_question` with the reason), so "the agent declined" is auditable rather than a hush.

---

12. Running it

```
pip install -r requirements.txt              # fastapi, uvicorn, pydantic, httpx, numpy, Pillow,
                                             # python-dotenv (scripts read .env themselves)
cp .env.example .env                         # every value optional for the offline path

python -m pytest tests -q                    # 176 passed + 1 xfailed, ~8 s, no network
python -u evals/run_evals.py                 # the four invariants -> evals/results.json, PASS
python -u evals/run_evals.py --llm off       # same invariants with the model dead
python backend/scripts/run_pack_b.py --llm off           # Pack B with the model dead
python backend/scripts/run_pack_b.py --llm replay        # recorded model turns, no network
python backend/scripts/run_pack_a.py --asr cached --llm replay
python backend/scripts/compare_runs.py openrouter record replay   # spends quota: records, then replays

RUN_MODE=cached-demo python -m uvicorn backend.app.server:app --port 8123   # the artifact
RUN_MODE=live-demo   python -m uvicorn backend.app.server:app --port 8127   # records real turns
python backend/scripts/drive_http.py --base http://127.0.0.1:8127 --pack gap --scenario S1 \
    --log backend/out/http_S1_live.jsonl
python backend/scripts/diff_frames.py backend/out/http_S1_live.jsonl \
    backend/out/http_S1_replay.jsonl           # proves the two are the same run

npm install && npm run build                 # frontend/dist, served by FastAPI at /
docker build -t dre-agent . && docker run -p 7860:7860 -e RUN_MODE=cached-demo dre-agent
python backend/scripts/make_technical_pdf.py # regenerates this .html + .pdf (pandoc + headless Chrome)
```

`RUN_MODE=live-demo` is the only mode that talks to a provider, and it writes what it hears into `tests/cassettes/`; `Cassette.save` overwrites on a key collision, so point `LLM_CASSETTE_DIR` at a scratch directory if you want a live run that cannot touch the committed store (§7.1).

`DEPLOY.md` covers publishing it to a public no-login link (Hugging Face Spaces with this Dockerfile, Render or Fly), which env vars a deployed instance needs, and the one command that proves the deployed link works (`drive_http.py` against the public URL — it exits non-zero if the fan would have been left with silence).

---

13. What this design is for

The interesting property of this system is not that an LLM wrote a match summary — any prompt does that. It is that **the summary can be audited line by line by someone who does not trust it.** Every number on the receipt traces to a feed event or a deterministic tally; every badge traces to a verdict computed in code; every boundary traces to a heartbeat row or to the fan's own words; every refusal — a clip rejected on a pixel disagreement, a spoiler citation, a skip overruled, a budget wall — is an event in a journal the reviewer can read. That is what makes the agent safe to put in front of a live match, and it is why the same kernel can carry a second, structurally different pack without changing a line of it.